

Compiler Construction (CS-4031)

Date: 06/03/2026

Course Instructors

Shaharbano Asif, Muhammad Kashif, Zulfiqar Ali, Ghulam Murtaza

Final Exam

Total Time: 03 Hrs

Total Marks: 100

Total Questions: 05

Roll No _____
Do not write below this line.

Section _____

Student Signature _____

Objective Part (Time 30 mins, Marks 20)

Question 1. Choose the best option.

Instructions:

1. To be attempted on the question papers.
2. Multiple marked answer will be considered wrong.
3. Pencils are not allowed.
4. Encircle your choice

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

1. Lexical error handling typically involves: ☒ a) Panic mode recovery b) Propagating the error to the parser c) Ignoring the entire source file d) Generating assembly code directly	5. Which of the following grammars is NOT suitable for top-down parsing? a) $S \rightarrow Aa \mid b, A \rightarrow c$ ☒ b) $S \rightarrow Sa \mid b$ c) $S \rightarrow aS \mid \epsilon$ d) $S \rightarrow aA, A \rightarrow b$
2. Runtime environment management (activation records) is primarily handled during: ☒ a) Code generation b) Lexical analysis c) Syntax analysis d) Intermediate code generation	6. An LR(0) item $A \rightarrow \alpha \cdot \beta$ indicates: ☒ a) The parser has recognized α and expects β b) The parser has finished parsing the whole production c) An error has occurred d) A shift action must be performed
3. A compiler that compiles itself is called: ☒ a) Self-hosting b) Bootstrapping c) Cross compiler d) Source-to-source compile	7. Which parser can parse $S \rightarrow Sa \mid a$? a) LL(1) b) SLR ☒ c) LR(1) d) LR(0)
4. Given input ifx and patterns: if (keyword), [a-z][a-z0-9]* (identifier), the token returned is: a) Keyword if ☒ b) Identifier ifx c) Error d) Both tokens	8. A reduce action in bottom-up parsing corresponds to: a) Replacing a non-terminal with a handle ☒ b) Replacing a handle with the corresponding non-terminal c) Shifting a terminal onto the stack d) Accepting the input string

9. For grammar $S \rightarrow L=R \mid R, L \rightarrow *R \mid id, R \rightarrow L$, an SLR parser will have: ☒ a) Shift-reduce conflict b) Reduce-reduce conflict c) No conflicts d) Only shift actions	15. The DAG for the expression $a + a * (b - c) + (b - c) * d$ will have how many nodes for the subexpression $(b - c)$? ☒ a) 1 b) 2 c) 3 d) 4
10. The action reduce by $A \rightarrow \beta$ in LR parsing means: ☒ a) Pop β symbols, push A b) Push β onto stack c) Shift β onto stack d) Goto new state without popping	16. "Strength Reduction" refers to: ☒ a) Replacing a complex operation with a cheaper one b) Removing dead code c) Folding constant expressions d) Inline expansion ? ^.
11. LR(1) items have lookahead symbols that are used: ☒ a) Only when a reduce is possible b) Only during shift c) For both shift and reduce d) Only for the start symbol	17. Constant folding involves: ☒ a) Replacing a multiplication by a shift b) Computing the value of constant expressions at compile time c) Removing unreachable code d) Renaming registers
12. Which of the following is an example of an Inherited Attribute? a) The value of a constant expression (computed bottom-up) ☒ b) The type of a variable passed down to its declaration c) The final result of an arithmetic expression d) The list of tokens	18. Copy propagation $t1 = t2; z = t1 + t3 \rightarrow z = t2 + t3$ requires: a) No change to $t2$ between copy and use ☒ b) $t2$ must be constant c) $t1$ must be dead d) Always invalid
13. Three-Address Code (3AC) is commonly represented as: ☒ a) A sequence of quadruples or triples b) A single complex expression c) A binary tree d) Assembly mnemonics directly	19. Given 3AC: $t1 = a + b$, a possible assembly translation (for a simple CPU) is: ☒ a) LOAD R1, a; LOAD R2, b; ADD R3, R1, R2; STORE t1, R3 b) ADD a, b, t1 c) $t1 = a + b$ (keep as is) d) MOV t1, a+b
14. Which three-address code statement is used for conditional jumps? a) goto L ☒ b) if $x < y$ goto L c) $x = y + z$ d) param x	20. The main challenge in converting 3AC to assembly for modern CPUs is: ☒ a) Managing a limited number of registers b) Parsing the 3AC c) Building the parse tree d) Tokenizing the 3AC